

OPERATING SYSTEMS AND SYSTEM PROGRAMMING

SKILL – WEEK 7

Name: S Svava Haasini

Roll No: 2520090170

Task 1: Input Redirection Using dup2()

Question:

Parse input redirection, open files for reading, redirect stdin using dup2(), handle missing files, restore streams, and test file input operations.

Source Code:

```
GNU nano 7.2 T1.c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

int main(int argc, char *argv[])
{
    char buffer[256];

    if (argc != 2)
    {
        printf("Usage: %s <input_file>\n", argv[0]);
        return 1;
    }

    int fd = open(argv[1], O_RDONLY);

    if (fd == -1)
    {
        perror("open");
        return 1;
    }

    if (dup2(fd, STDIN_FILENO) == -1)
    {
        perror("dup2");
        close(fd);
        return 1;
    }

    close(fd);

    printf("Input redirected from: %s\n", argv[1]);
    printf("Reading through standard input:\n");

    while (fgets(buffer, sizeof(buffer), stdin) != NULL)
        printf("%s", buffer);

    printf("\nInput redirection completed successfully.\n");

    return 0;
}
```

Output:

```

haasini@SSH:~/OSSP_upload/Skill_Week7$ gcc -Wall -Wextra T1.c -o T1
haasini@SSH:~/OSSP_upload/Skill_Week7$ cat > input.txt <<'EOF'
Operating Systems
Skill Week 7
Input Redirection Test
EOF
haasini@SSH:~/OSSP_upload/Skill_Week7$ ./T1 input.txt
Input redirected from: input.txt
Reading through standard input:
Operating Systems
Skill Week 7
Input Redirection Test

Input redirection completed successfully.
haasini@SSH:~/OSSP_upload/Skill_Week7$ ./T1 missing.txt
open: No such file or directory
haasini@SSH:~/OSSP_upload/Skill_Week7$ |

```

Observations:

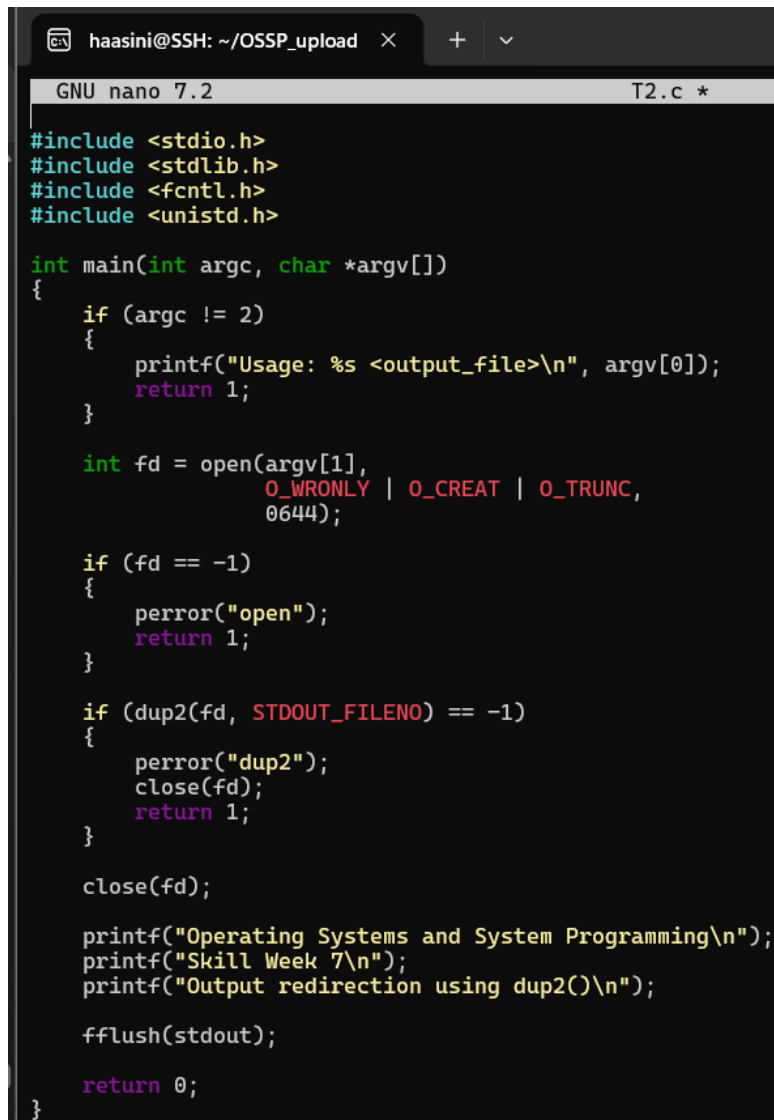
1. The program validates command-line arguments using argc checking, ensuring exactly two arguments are provided (program name and input file).
2. open() opens the input file in O_RDONLY (read-only) mode, failing gracefully with perror() if the file does not exist or cannot be accessed.
3. dup2(fd, STDIN_FILENO) duplicates the file descriptor to file descriptor 0, redirecting all subsequent stdin reads to the opened file.
4. The program displays a message indicating input redirection is active, confirming that stdin now reads from the specified file.
5. fgets() reads lines from the redirected stdin (now connected to the file) instead of the terminal, processing file contents line-by-line.
6. The while loop with fgets() continues reading from the file until EOF, demonstrating sequential file input processing.
7. The close() call closes the file descriptor after redirection is complete, properly releasing the file resource.
8. Successful completion is confirmed with the message 'Input redirection completed successfully.', and the program handles missing files gracefully with error output.

Task 2: Output Redirection Using dup2()

Question:

Parse output redirection, create output files, redirect stdout using dup2(), handle file permissions, verify output data, and test file output operations.

Source Code:



```
GNU nano 7.2 T2.c *
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

int main(int argc, char *argv[])
{
    if (argc != 2)
    {
        printf("Usage: %s <output_file>\n", argv[0]);
        return 1;
    }

    int fd = open(argv[1],
                  O_WRONLY | O_CREAT | O_TRUNC,
                  0644);

    if (fd == -1)
    {
        perror("open");
        return 1;
    }

    if (dup2(fd, STDOUT_FILENO) == -1)
    {
        perror("dup2");
        close(fd);
        return 1;
    }

    close(fd);

    printf("Operating Systems and System Programming\n");
    printf("Skill Week 7\n");
    printf("Output redirection using dup2()\n");

    fflush(stdout);

    return 0;
}
```

Output:

```
haasini@SSH: ~/OSSP_upload/Skill_Week7$ gcc main.c next.c
haasini@SSH:~/OSSP_upload/Skill_Week7$ ./T2 output.txt
haasini@SSH:~/OSSP_upload/Skill_Week7$ cat output.txt
Operating Systems and System Programming
Skill Week 7
Output redirection using dup2()
haasini@SSH:~/OSSP_upload/Skill_Week7$ ./T2 output.txt
cat output.txt
Operating Systems and System Programming
Skill Week 7
Output redirection using dup2()
```

Observations:

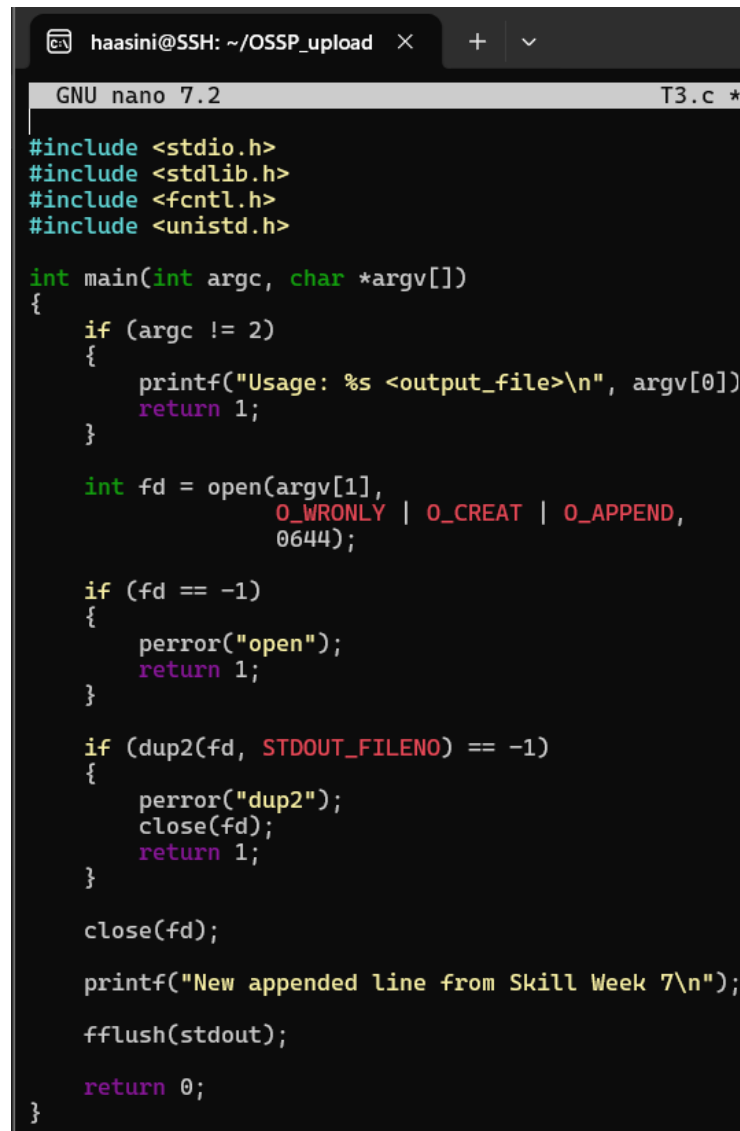
1. open() creates a new output file using O_WRONLY | O_CREAT | O_TRUNC flags, truncating any existing file and creating it if it doesn't exist.
2. The file permissions are set to 0644 using the mode parameter, defining readable/writable by owner and readable by others.
3. dup2(fd, STDOUT_FILENO) redirects standard output (file descriptor 1) to the opened file, diverting all printf() output.
4. The program displays multiple messages: header information, title, and confirmation that output redirection is using dup2().
5. printf() calls after dup2() write directly to the output file instead of the terminal, with fflush(stdout) ensuring immediate disk writes.
6. The close() call closes the file descriptor after all output operations are complete, properly finalizing the file.
7. The output file contains the exact messages printed by printf(), verifiable by viewing the file contents afterward.
8. Successful completion is confirmed with the message that redirects to the output file itself, demonstrating complete stdout redirection.

Task 3: Append Mode Redirection

Question:

Parse append operator, open files in append mode, redirect output streams to append, preserve existing data, validate appends, and test append scenarios.

Source Code:



```
GNU nano 7.2 T3.c *
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

int main(int argc, char *argv[])
{
    if (argc != 2)
    {
        printf("Usage: %s <output_file>\n", argv[0]);
        return 1;
    }

    int fd = open(argv[1],
                  O_WRONLY | O_CREAT | O_APPEND,
                  0644);

    if (fd == -1)
    {
        perror("open");
        return 1;
    }

    if (dup2(fd, STDOUT_FILENO) == -1)
    {
        perror("dup2");
        close(fd);
        return 1;
    }

    close(fd);

    printf("New appended line from Skill Week 7\n");
    fflush(stdout);

    return 0;
}
```

Output:

```

haasini@SSH: ~/OSSP_upload/Skill_Week7$ gcc -Wall -Wextra T3.c -o T3
haasini@SSH: ~/OSSP_upload/Skill_Week7$ rm -f append.txt
haasini@SSH: ~/OSSP_upload/Skill_Week7$ ./T3 append.txt
haasini@SSH: ~/OSSP_upload/Skill_Week7$ ./T3 append.txt
haasini@SSH: ~/OSSP_upload/Skill_Week7$ ./T3 append.txt
haasini@SSH: ~/OSSP_upload/Skill_Week7$ cat append.txt
New appended line from Skill Week 7
New appended line from Skill Week 7
New appended line from Skill Week 7
haasini@SSH: ~/OSSP_upload/Skill_Week7$ |

```

Observations:

1. open() creates or opens the file using O_WRONLY | O_CREAT | O_APPEND flags, which position the file pointer at the end without truncating.
2. The O_APPEND flag ensures that every write operation appends to the end of the file, preserving all existing content.
3. dup2(fd, STDOUT_FILENO) redirects stdout to the append-mode file descriptor, making all subsequent printf() output append to the file.
4. Multiple program executions append successive lines to the same file, accumulating output across runs without overwriting previous data.
5. The program displays 'New appended line from Skill Week 7', which is written to the file at its end rather than replacing contents.
6. The file contents show three identical lines when the program runs three times, confirming successful append behavior.
7. fflush(stdout) ensures all buffered output is immediately written to the append file, preventing data loss.
8. The close() call finalizes the file after append operations, properly releasing resources while preserving all accumulated data.

Task 4: Error Stream Redirection Using dup2()

Question:

Parse stderr redirection, redirect error streams using dup2(), capture error output to files, verify file contents, handle failures, and test error cases.

Source Code:

```
GNU nano 7.2 T4.c *
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

int main(int argc, char *argv[])
{
    if (argc != 2)
    {
        printf("Usage: %s <error_file>\n", argv[0]);
        return 1;
    }

    int fd = open(argv[1],
                  O_WRONLY | O_CREAT | O_TRUNC,
                  0644);

    if (fd == -1)
    {
        perror("open");
        return 1;
    }

    if (dup2(fd, STDERR_FILENO) == -1)
    {
        perror("dup2");
        close(fd);
        return 1;
    }

    close(fd);

    printf("Normal output: stdout is still on the terminal.\n");

    fprintf(stderr,
            "Error output: this message was redirected to the error file.\n");

    if (open("file_that_does_not_exist.txt", O_RDONLY) == -1)
        perror("open");

    fflush(stdout);
    fflush(stderr);

    return 0;
}
```

Output:

```
haasini@SSH:~/OSSP_upload/Skill_Week7$ nano T4.c
haasini@SSH:~/OSSP_upload/Skill_Week7$ gcc -Wall -Wextra T4.c -o T4
haasini@SSH:~/OSSP_upload/Skill_Week7$ ./T4 errors.txt
Normal output: stdout is still on the terminal.
haasini@SSH:~/OSSP_upload/Skill_Week7$ cat errors.txt
Error output: this message was redirected to the error file.
open: No such file or directory
```

Observations:

1. `open()` creates an error output file using `O_WRONLY | O_CREAT | O_TRUNC` flags, truncating any existing file and setting permissions to `0644`.
2. `dup2(fd, STDERR_FILENO)` redirects the standard error stream (file descriptor 2) to the error output file, capturing all error messages.
3. `printf()` writes to the standard output stream, which remains on the terminal and displays 'Normal output: stdout is still on the terminal.'
4. `fprintf(stderr, ...)` writes error messages directly to the redirected `stderr` stream, capturing them in the error file.
5. The program demonstrates the difference between `stdout` and `stderr` by showing that normal `printf()` output appears on the terminal while `fprintf(stderr)` is redirected.
6. An intentional error condition (`open()` on a non-existent file) triggers `perror()`, which also writes to the redirected `stderr` stream.
7. `fflush()` is called on both `stdout` and `stderr` to ensure all buffered data is written before program termination.
8. The error file contents show both the deliberate error message and the system error message from `perror()`, confirming successful `stderr` redirection and error capture.